

NutriGen (F24-075-D)

Group Members

Hamza Shahid (21I-0815)

Kashan Ali (20I-2421)

Hammad Kabir (20I-2480)

Supervised by

Ms.Nirmal Tariq



Department of Computer Science

National University of Computer and Emerging Sciences

Islamabad, Pakistan

2024

Table Of Contents

Chapter 1.....	4
Introduction.....	4
1.1 Problem Statement.....	4
1.2 Scope.....	4
1.3 Modules.....	5
1.3.1 Genetic Data Integration Module.....	5
1.3.2 Meal Planning Module.....	6
1.3.3 Goal Tracking Module.....	6
1.3.4 Allergy Management Module.....	6
1.3.5 Health Dashboard Module.....	6
1.3.6 Reporting Module.....	6
1.3.7 Nutritionist Platform Module.....	7
1.3.8 Gamification Module.....	7
1.4 Use Cases:.....	7
1.4.1 Use Case 1: Upload Genetic Data.....	7
1.4.2 Use Case 2: Generate Personalized Meal Plan.....	8
1.4.3 Use Case 3: Track Daily Food Intake.....	9
1.4.4 Use Case 4: View Health Dashboard.....	10
1.4.5 Use Case 5: Consult with Nutritionist.....	10
1.5 Use Case Diagram for NutriGen:.....	11
Figure 1.5.1 Use Case Diagram.....	12
1.6 User Classes and Characteristics.....	12
Chapter 2.....	13
2.1 Functional Requirements.....	14
2.1.1 Genetic Data Integration Module.....	14
2.1.2 Meal Planning Module.....	14
2.1.3 Goal Tracking Module.....	14
2.1.4 Allergy Management Module.....	14
2.1.5 Health Dashboard Module.....	15
2.1.6 Reporting Module.....	15
2.1.7 Nutritionist Platform Module.....	15
2.1.8 Gamification Module.....	15
2.2 Non-Functional Requirements.....	15
2.2.1 Usability.....	16
2.2.2 Performance.....	16
2.2.3 Security.....	16

2.3 Domain Model For NutriGen:.....	17
Figure 2.3.1 Domain Model Diagram.....	18
2.4 Event Response Table.....	18
Table 2.4.1.....	20
Chapter 3.....	21
System Overview.....	21
3.1 Architectural Design.....	21
Figure 3.1.1 Architecture Diagram.....	22
3.2 Design Models.....	23
3.2.1 Activity Diagrams.....	24
Figure 3.2.1.1 Activity Diagram.....	25
Figure 3.2.1.2 Activity Diagram.....	26
Figure 3.2.1.1 Activity Diagram.....	28
3.2.2 Class Diagram.....	29
Figure 3.2.2.1 Class Diagram.....	29
3.2.3 System Sequence Diagrams.....	31
Figure 3.2.3.1 System Sequence Diagram.....	31
Figure 3.2.3.2 System Sequence Diagram.....	32
Figure 3.2.3.3 System Sequence Diagram.....	33
Figure 3.2.3.4 System Sequence Diagram.....	33
3.2.4 State Transition Diagram.....	34
Figure 3.2.4.1 State Transition Diagram.....	34
3.3 Data Design.....	35
3.3.1 Data Structures Overview.....	35
3.3.1.1 User Data.....	35
3.3.1.2 Genetic Profile Data.....	35
3.3.1.3 Meal Plan Data.....	36
3.3.1.4 Food Item Data.....	36
3.3.1.5 Health Goal Data.....	36
3.3.1.6 Health Metric Data.....	36
3.3.1.7 Nutritionist Profile Data.....	36
3.3.2 Data Storage Solutions.....	37
3.3.3 Data Flow and Processing.....	38
3.3.4 Data Security and Privacy.....	39
3.3.5 Entity Relationship Diagram.....	39
Figure 3.3.5.1 ERD.....	41
Chapter 4: Conclusions and Future Work.....	42
4.1 Project Summary.....	42
4.2 Next Steps.....	42
4.3 Future Ideas.....	43
4.4 Closing Thoughts.....	43

Chapter 1

Introduction

This document talks about what NutriGen needs to do. NutriGen is a phone app that helps people eat better by looking at their genes. This document is for the people making the app, like the people who write the code, the people who manage the project, and the people who check if the app works right.

1.1 Problem Statement

Eating right is hard for many people. This is because the advice about food that works for one person might not work for another. Our bodies are all different, and this is partly because of our genes. Genes are like instructions in our body that tell it how to use food. But most people don't know how to use information about their genes to eat better.

NutriGen helps solve this problem. It looks at a person's gene information and uses it to make a special eating plan just for them. The app does more than just make meal plans. It also helps people keep track of how they're doing, tells them what foods might make them sick, and helps them talk to food experts. By doing all this, NutriGen helps people eat in a way that's just right for their own body. This can help them be healthier and feel better.

1.2 Scope

NutriGen is an app for phones that uses smart computer programs to give people advice about what to eat. Here's what the app does:

First, people can put their gene test results into the app. This is like giving the app special information about their body. The app then looks at this information very carefully. It figures out what kinds of food are best for that person. After that, it makes a meal plan that fits with the person's genes and what they want to achieve with their health. If people don't like the meal plan, they can change it.

The app also watches what the person eats every day. It keeps track of how close they are to reaching their health goals. If the app thinks a food might make the person sick (based on their gene information), it warns them. There's also a part of the app that shows people how healthy they are overall, like a health report card.

Every month, the app writes a report. This report shows how the person is doing and gives them ideas about how they could do even better. The app also lets people find food experts. They can see what other people think about these experts. To make using the app more fun, it gives people badges when they reach their health goals, like a game.

All of these things are meant to help people eat in a way that's best for their own unique body. The app uses science to figure this out, so people can trust that the advice is good for them.

1.3 Modules

The app has different parts that do different jobs. Here are the main parts:

1.3.1 Genetic Data Integration Module

This part of the app deals with the user's gene information.

- It lets users put their gene test results into the app
- It looks at the gene information to find things about food and health
- It tells users what their genes say about their food needs

1.3.2 Meal Planning Module

This part makes and takes care of personal meal plans.

- It makes meal plans based on what it learned from the user's genes
- It lets users change their meal plans if they want to
- It suggests other meals that are also good for the user

1.3.3 Goal Tracking Module

This part helps users see how well they're doing.

- It lets users set goals for their health and eating
- It keeps track of what users eat each day
- It shows users how close they are to reaching their goals

1.3.4 Allergy Management Module

This part helps users avoid foods that might make them sick.

- It uses gene information to guess what foods might cause problems
- It changes meal plans to avoid foods that might be bad for the user
- It warns users about foods that might make them sick

1.3.5 Health Dashboard Module

This part shows users how healthy they are.

- It shows what kinds of nutrients the user is eating
- It shows how much the user is moving and exercising
- It shows other important health information

1.3.6 Reporting Module

This part makes reports about the user's health.

- It makes a report each month to show how the user is doing
- It gives personal advice about how to be healthier
- It shows how the user's health is changing over time

1.3.7 Nutritionist Platform Module

This part helps users talk to food experts.

- It shows users information about different food experts
- It lets users say what they think about the food experts
- It helps users set up times to talk to food experts

1.3.8 Gamification Module

This part makes the app more fun to use.

- It gives users badges when they reach health goals
- It sets up a system for earning and showing badges
- It makes fun challenges to keep users interested

1.4 Use Cases:

1.4.1 Use Case 1: Upload Genetic Data

- **Scope:** Genetic Data Integration Module
- **Level:** User goal
- **Primary Actor:** User
- **Stakeholders and Interests:**
 - **User:** Wants to upload genetic data securely and easily
 - **System:** Needs accurate genetic data for analysis

- **Nutritionists:** Require accurate genetic information to provide tailored advice

Preconditions: Users must be logged in and have genetic test results available.

Success Guarantee: Genetic data is uploaded and stored securely in the system.

Main Success Scenario:

1. User selects the "Upload Genetic Data" option.
2. System prompts the user to select a genetic data file.
3. User selects and confirms the file.
4. System verifies file format and uploads the data.
5. System confirms successful upload and notifies the user.

Extensions:

User uploads incorrect file type:

1. System notifies the user of error.
2. Return to the previous step.

Upload fails due to network issues:

1. System notifies the user of failure.
2. Prompts users to try again.

1.4.2 Use Case 2: Generate Personalized Meal Plan

- **Scope:** Meal Planning Module
- **Level:** System goal
- **Primary Actor:** System
- **Secondary Actor:** User

Preconditions: User's genetic data and health goals are in the system.

Success Guarantee: A personalized meal plan is generated based on the user's genetic data and goals.

Main Success Scenario:

1. System analyzes the user's genetic data.
2. System considers the user's health goals and preferences.
3. System generates a meal plan tailored to the user.
4. System presents the meal plan to the user.
5. User reviews and accepts the meal plan.

Extensions:

User is not satisfied with the meal plan:

1. User requests modifications.
2. System adjusts the meal plan.
3. Return to the previous step.

1.4.3 Use Case 3: Track Daily Food Intake

- **Scope:** Goal Tracking Module

- **Level:** User goal

- **Primary Actor:** User

Preconditions: User has an active meal plan.

Success Guarantee: User's daily food intake is recorded and progress towards goals is updated.

Main Success Scenario:

1. User selects the "Log Food" option.
2. System displays a food logging interface.
3. User enters food items consumed.
4. System records the data and updates the user's progress.
5. System displays updated progress towards goals.

Extensions:

User can't find a specific food item:

1. User selects the "Add Custom Food" option.
2. User enters food details.

3. System adds the custom food to the database.
4. Return to the previous step.

1.4.4 Use Case 4: View Health Dashboard

- **Scope:** Health Dashboard Module
- **Level:** User goal
- **Primary Actor:** User

Preconditions: User has been using the app and logging data.

Success Guarantee: User views a comprehensive health dashboard with current metrics and trends.

Main Success Scenario:

1. User selects the "Health Dashboard" option.
2. System retrieves the user's health data.
3. System generates visualizations of health metrics and trends.
4. System displays the dashboard to the user.
5. User reviews the information.

Extensions:

User requests more detailed information on a specific metric:

1. User selects the metric.
2. System displays detailed breakdown and historical data.
3. Return to the previous step.

1.4.5 Use Case 5: Consult with Nutritionist

- **Scope:** Nutritionist Platform Module
- **Level:** User goal
- **Primary Actor:** User
- **Secondary Actor:** Nutritionist

Preconditions: User is logged in and nutritionist profiles are available.

Success Guarantee: User successfully books an appointment with a nutritionist.

Main Success Scenario:

1. User selects the "Find Nutritionist" option.
2. System displays a list of available nutritionists.
3. User reviews nutritionist profiles and selects one.
4. System shows nutritionist's available time slots.
5. User selects a time slot and confirms booking.
6. System notifies the nutritionist and confirms the appointment with the user.

Extensions:

User wants to filter nutritionists by specialty:

1. User applies a filter.
2. System updates the list.
3. Return to the previous step.

Selected time slot becomes unavailable:

1. System notifies the user and shows alternative slots.
2. Return to the previous step.

These use cases provide a more detailed view of how users will interact with the NutriGen system, following a similar pattern to the InsightVision document. Each use case includes the scope, level, actors, preconditions, success guarantees, main scenarios, and possible extensions, giving a comprehensive overview of the system's functionality.

1.5 Use Case Diagram for NutriGen:

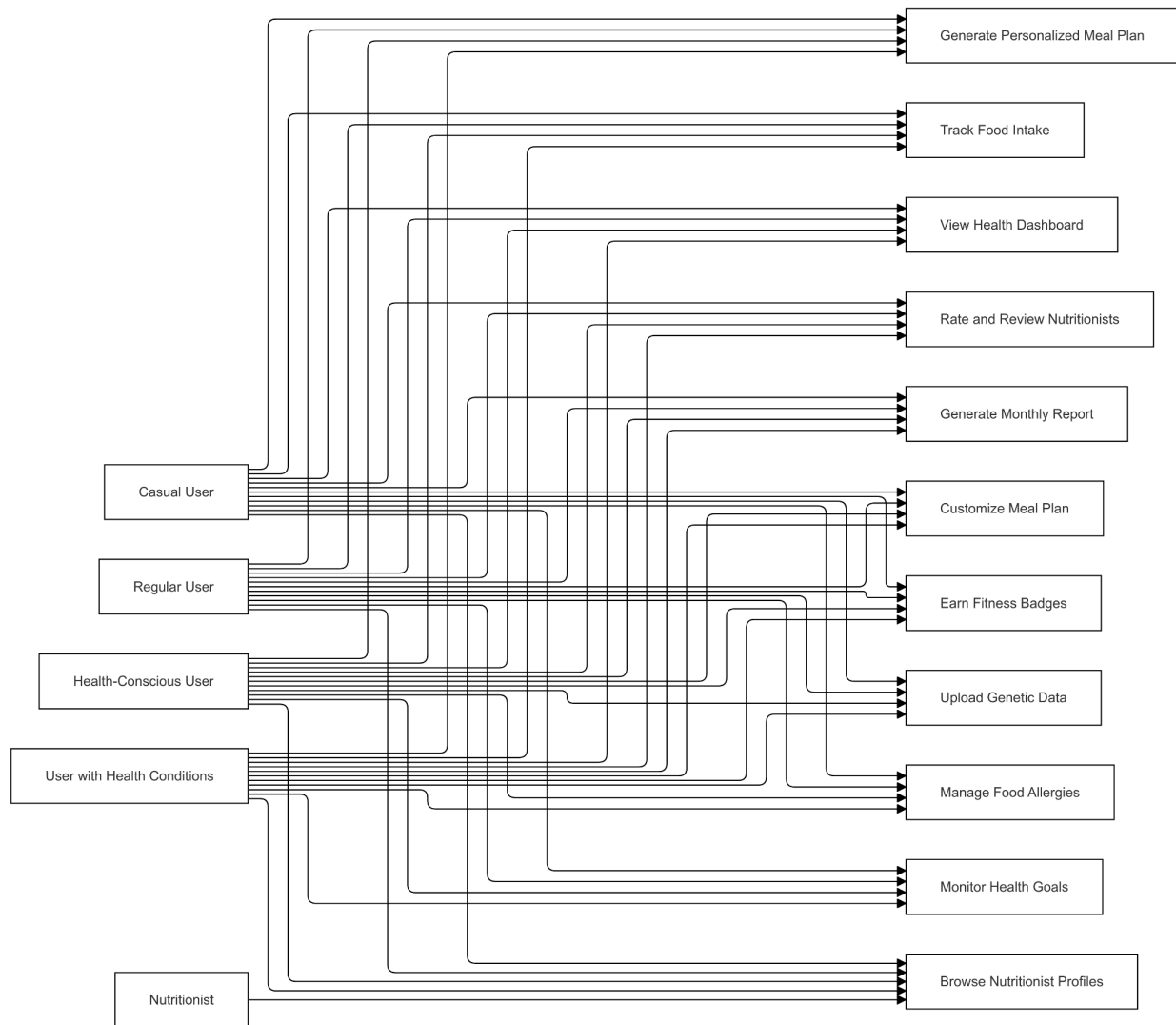


Figure 1.5.1 Use Case Diagram

1.6 User Classes and Characteristics

Different kinds of people will use the app. Here are the main types:

Regular Users:

These are the most common users of NutriGen. They want to eat better and they're interested in using their gene information to do it. They'll probably use the app every

day to look at their meal plans, keep track of what they eat, and see how they're doing. They might not know much about genes or nutrition when they start. They'll probably use most parts of the app.

Health-Conscious Users:

These users already know quite a bit about health and nutrition. They want to use their gene information to make their diet even better. They'll probably use the more complicated parts of the app. They might change their meal plans more often than regular users.

Users with Health Conditions:

These users have specific health problems, like diabetes or heart issues. They need meal plans that help them manage their health. They'll pay extra attention to certain parts of the health dashboard that are important for their condition.

Nutritionists:

These are food and health experts. The app will show information about them, like their experience and what they're good at. Users can look at nutritionist profiles and contact them if they need help. Nutritionists can pay to have their profiles shown to more users.

Casual Users:

These users are just starting to think about eating better. They might not use all of the app's features. They'll mostly look at simple meal plans and general health tips.

Chapter 2

2.1 Functional Requirements

This section talks about what the app needs to do. We split it up based on the different parts of the app.

2.1.1 Genetic Data Integration Module

FR1: The app must let users put in their gene test results.

FR2: The app must look at the gene information to find things about food and health.

FR3: The app must tell users what their genes say about their food needs.

2.1.2 Meal Planning Module

FR4: The app must make meal plans based on the user's gene information.

FR5: The app must let users change their meal plans.

FR6: The app must suggest other meals that are also good for the user.

2.1.3 Goal Tracking Module

FR7: The app must let users set goals for their health and eating.

FR8: The app must keep track of what users eat each day.

FR9: The app must show users how close they are to reaching their goals.

2.1.4 Allergy Management Module

FR10: The app must use gene information to guess what foods might cause problems.

FR11: The app must change meal plans to avoid foods that might be bad for the user.

FR12: The app must warn users about foods that might make them sick.

2.1.5 Health Dashboard Module

FR13: The app must show what kinds of nutrients the user is eating.

FR14: The app must show how much the user is moving and exercising.

FR15: The app must show other important health information.

2.1.6 Reporting Module

FR16: The app must make a report each month to show how the user is doing.

FR17: The app must give personal advice about how to be healthier.

FR18: The app must show how the user's health is changing over time.

2.1.7 Nutritionist Platform Module

FR19: The app must show users information about different food experts.

FR20: The app must let users say what they think about the food experts.

FR21: The app must help users set up times to talk to food experts.

2.1.8 Gamification Module

FR22: The app must give users badges when they reach health goals.

FR23: The app must set up a system for earning and showing badges.

FR24: The app must make fun challenges to keep users interested.

2.2 Non-Functional Requirements

This section talks about how well the app needs to work.

2.2.1 Usability

NFR1: Users must be able to see their latest meal plan by tapping no more than two times from the main screen.

NFR2: New users must be able to set up their account and put in their gene information in no more than 10 minutes.

2.2.2 Performance

NFR3: The app must make a personal meal plan within 3 minutes after getting the user's gene information.

NFR4: The app must show the user's health dashboard within 3 seconds of opening the app.

2.2.3 Security

NFR5: The app must keep all user information, especially gene information, very safe using a special code system called AES-256.

NFR6: The app must ask users to log in again before they can change important settings or see sensitive health information.

2.3 Domain Model For NutriGen:

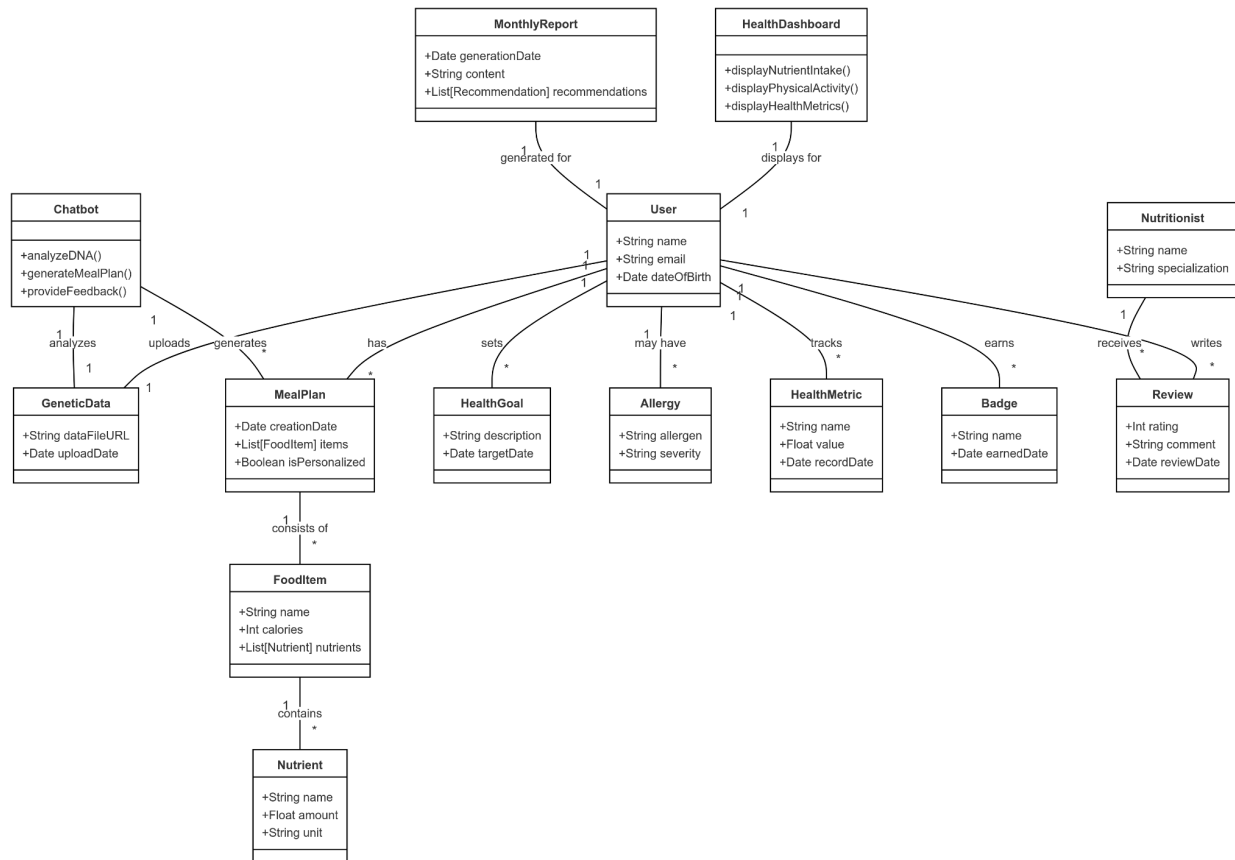


Figure 2.3.1 Domain Model Diagram

2.4 Event Response Table

Event	System Response
User uploads genetic data	1. Genetic Data Processing Module validates the data format 2. Data is securely stored in the database.

	3. Genetic analysis process is initiated to extract nutritional insights.
Genetic analysis completed	1. Results are stored in the user's profile. 2. Meal Planning Module is triggered to generate initial personalized meal plans.
User requests personalized meal plan	1. Meal Planning Module retrieves user's genetic insights and preferences. 2. An AI algorithm generates a tailored meal plan. 3. Plan is stored and sent to the user interface for display.
User logs food intake	1. Goal Tracking Module records the food data. 2. Nutritional values are calculated and added to the user's daily intake. 3. Progress towards health goals is updated.
Daily nutrition goal met/missed	1. Gamification Module is notified.
Monthly report generation time	1. The Reporting Module collects user's monthly data. 2. AI analyzes trends and generates insights. 3. Report is compiled and stored for user access.
User views nutritionist profile	1. Nutritionist Platform Module retrieves the selected nutritionist's profile information. 2. Profile details, including contact information, are displayed to the user.
Nutritionist boosts their profile	1. Nutritionist Platform Module updates the visibility settings for the nutritionist's profile. 2. Profile is marked for increased visibility in user searches. 3. Billing system is notified to process the

	boost fee.
User reaches a health milestone	1. Gamification Module is notified. 2. Appropriate badge is awarded. 3. Achievement is recorded in the user's profile.
System detects potential food allergy	1. Allergy Management Module updates user's profile. 2. Meal Planning Module is notified to adjust future meal suggestions. 3. Users are alerted about the potential allergy.
User updates health goals	1. Goal Tracking Module updates the user's profile. 2. Meal Planning Module adjusts meal plans to align with new goals. 3. The Health Dashboard is updated to reflect new targets.

Table 2.4.1

Chapter 3

System Overview

3.1 Architectural Design

NutriGen is built using different parts that work together. These parts help the app do its job of helping people eat better based on their genes. Here's how the app is put together:

1. User Interface:

- This is what people see on their phone screen.
- It lets users put in their information and see their meal plans.

2. Genetic Data:

- This part looks at the gene information users put in.
- It figures out what the gene information means for the user's diet.

3. Meal Planning:

- This part makes meal plans based on what the Genetic Data Part found out.
- It also lets users change their meal plans if they want to.

4. Goal Tracking:

- This part helps users set health goals and see how well they're doing.

5. Health Dashboard:

- This part shows users how healthy they are overall.
- It puts together information from other parts to make easy-to-understand pictures.

6. Nutritionist Platform:

- This part shows information about food experts (nutritionists).

7. Data Storage:

- This part keeps all the information safe and organized.
- It helps other parts get the information they need quickly.

The most suitable architectural pattern for NutriGen would be a Layered Architecture, specifically a three-tier architecture. This pattern is well-suited for mobile applications and aligns with the modular structure of NutriGen.

The three layers would be:

1. Presentation Layer (Front-End)
2. Application Layer (Backend)
3. Data Layer

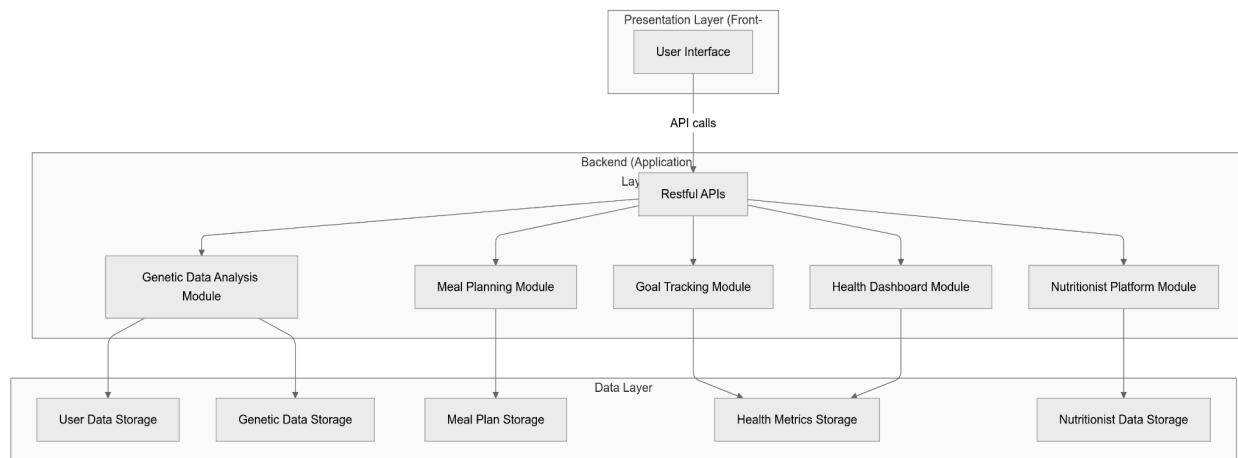


Figure 3.1.1 Architecture Diagram

In this diagram:

1. The Presentation Layer would contain the mobile app user interface.
2. The Application Layer would include the core functional modules of NutriGen.
3. The Data Layer would handle data storage and retrieval for various types of data used in the app.

3.2 Design Models

This Section includes all of the relevant design diagrams for NutriGen, giving an overview of the system architecture and behavior pattern using an Object Oriented Development Approach, and each diagram is provided with details and brief description to explain the purpose of each diagram.

For NutriGen, an Object-Oriented Programming (OOP) approach would be more suitable, because:

1. **Better Organization:** NutriGen has many different parts like users, meal plans, and genetic data. OOP lets us make each of these a separate "object". This makes the app easier to organize and understand.
2. **Reusable Code:** We can create a basic "plan" object and use it for different types of plans. This saves time and makes the code cleaner.
3. **Easier Updates:** If we need to change how meal plans work, we can just update the meal plan "object" without messing with other parts of the app.

3.2.1 Activity Diagrams

These three diagrams will cover the main user flows in NutriGen: initial setup, meal planning, and ongoing usage for health tracking. They provide a clear visual representation of how users will interact with the system and how the system will process and respond to user actions.

1. User Registration and Genetic Data Upload

This diagram covers the initial user interaction with NutriGen, including account creation and genetic data upload. It shows the flow from app opening to successful data processing.

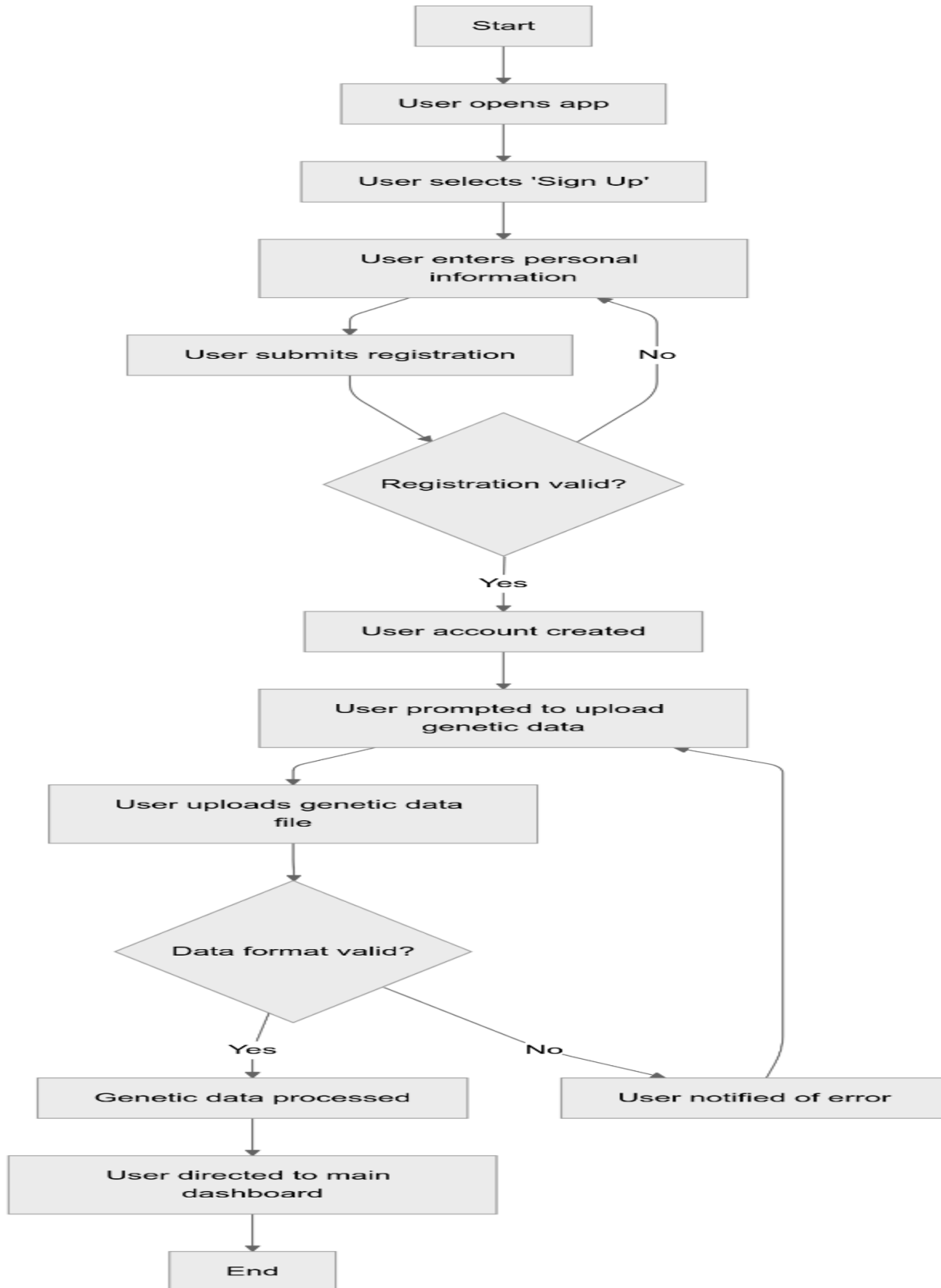


Figure 3.2.1.1 Activity Diagram

2.Meal Plan Generation and Customization

This diagram illustrates the process of creating and customizing a meal plan. It shows how the system uses genetic data to generate a plan and allows user customization.

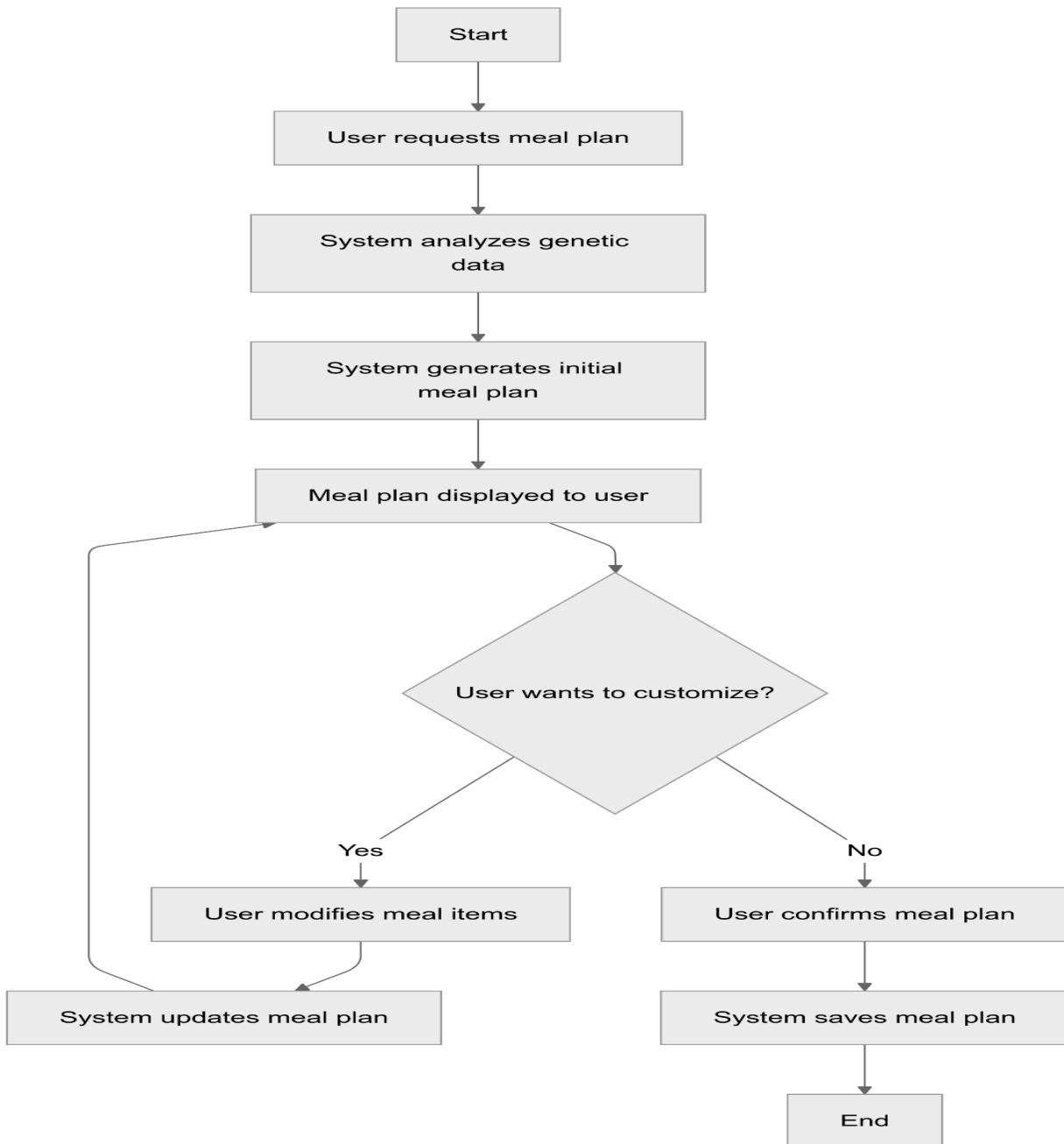


Figure 3.2.1.2 Activity Diagram

3. Daily Progress Tracking and Reporting

This diagram covers the daily user interaction for food logging and progress tracking, as well as the monthly reporting feature. It shows how the system continuously updates user progress and generates periodic reports.

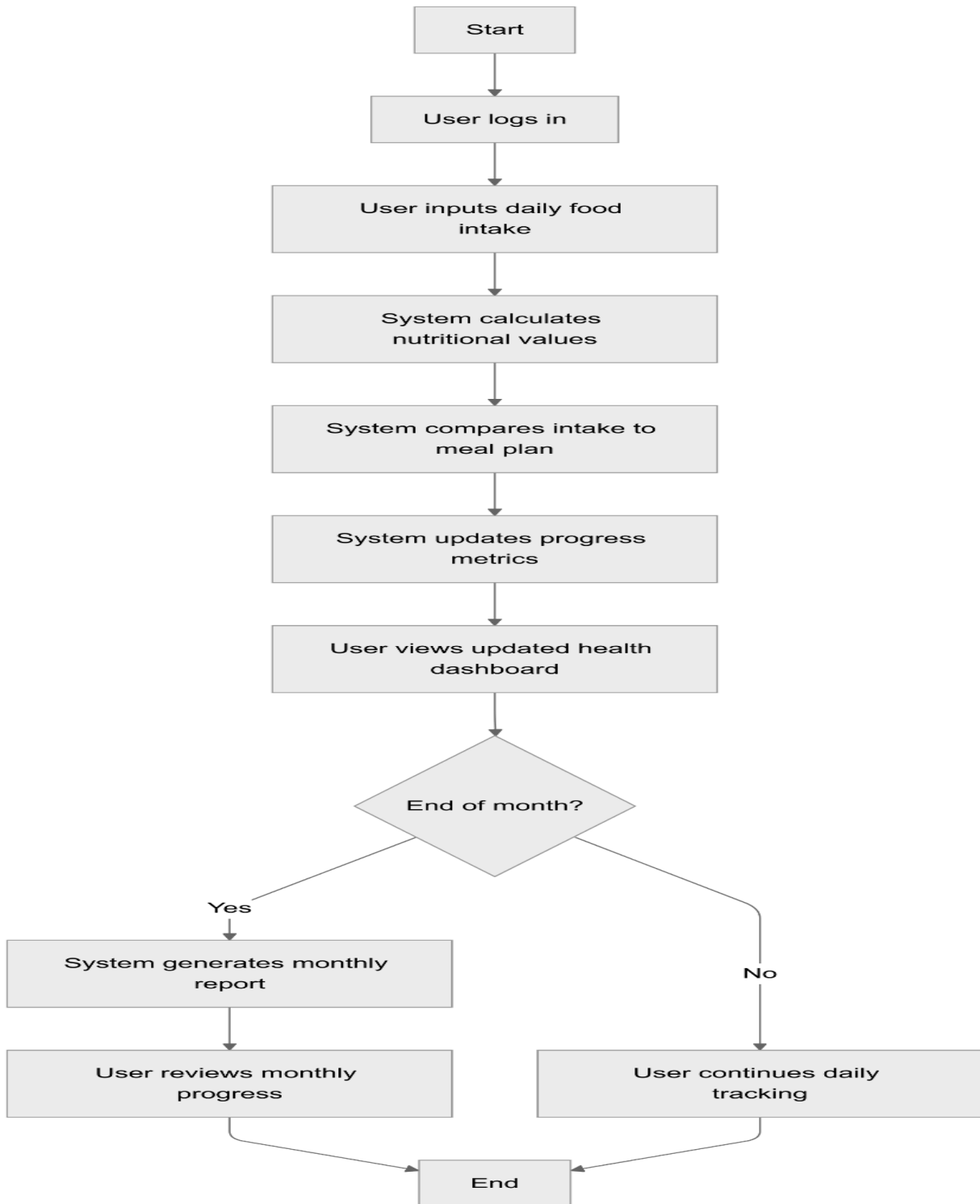


Figure 3.2.1.1 Activity Diagram

3.2.2 Class Diagram

The class diagram provides a solid foundation for implementing NutriGen using object-oriented programming principles. It captures the essential entities and their interactions, setting the stage for a modular, maintainable, and extensible system architecture. This is just sort of an overview, the original implementation may differ from this.

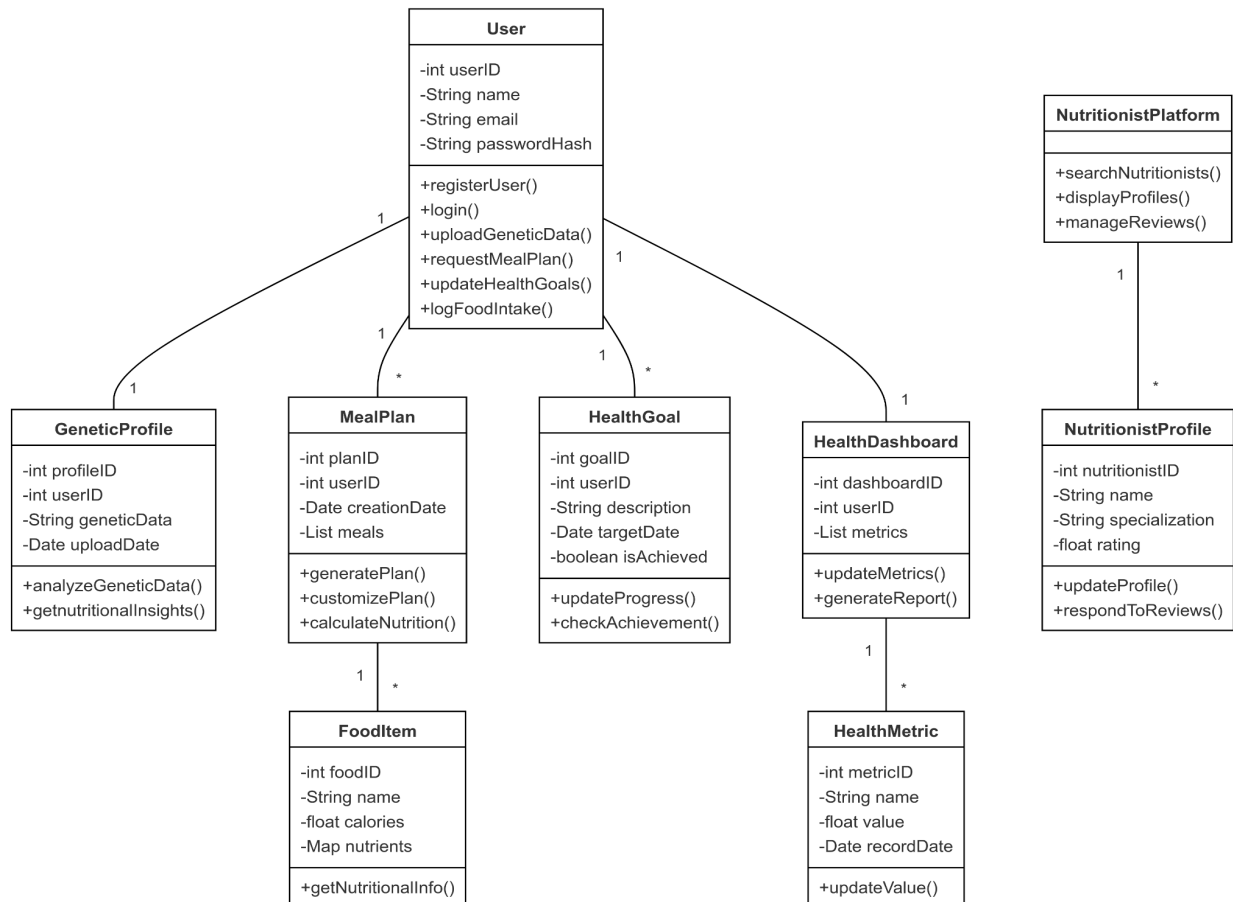


Figure 3.2.2.1 Class Diagram

Entities:

1. User: Central entity representing the app's users.
2. GeneticProfile: Stores user's genetic data.
3. MealPlan: Represents personalized meal plans.
4. FoodItem: Individual food items in meal plans.
5. HealthGoal: User's health objectives.
6. HealthDashboard: Displays user's health metrics.
7. HealthMetric: Individual health measurements.
8. NutritionistPlatform: Manages nutritionist information.
9. NutritionistProfile: Individual nutritionist details.

Relationships:

1. User - GeneticProfile: One-to-one. Each user has one genetic profile.
2. User - MealPlan: One-to-many. A user can have multiple meal plans.
3. User - HealthGoal: One-to-many. Users can set multiple health goals.
4. User - HealthDashboard: One-to-one. Each user has one health dashboard.
5. MealPlan - FoodItem: One-to-many. A meal plan consists of multiple food items.
6. HealthDashboard - HealthMetric: One-to-many. A dashboard displays multiple health metrics.
7. NutritionistPlatform - NutritionistProfile: One-to-many. The platform contains multiple nutritionist profiles.

Key Attributes:

- User: userID, name, email
- GeneticProfile: profileID, geneticData
- MealPlan: planID, creationDate
- FoodItem: foodID, name, calories
- HealthGoal: goalID, description, targetDate
- HealthMetric: metricID, name, value

Key Methods:

- User: uploadGeneticData(), requestMealPlan()
- GeneticProfile: analyzeGeneticData()
- MealPlan: generatePlan(), customizePlan()

- HealthDashboard: updateMetrics(), generateReport()
- NutritionistPlatform: searchNutritionists()

3.2.3 System Sequence Diagrams

1. User Registration and Genetic Data Upload

This shows the sequence of interactions when a new user registers and uploads their genetic data. It involves the User Interface, UserController, User class, GeneticProfile class, and the Database.

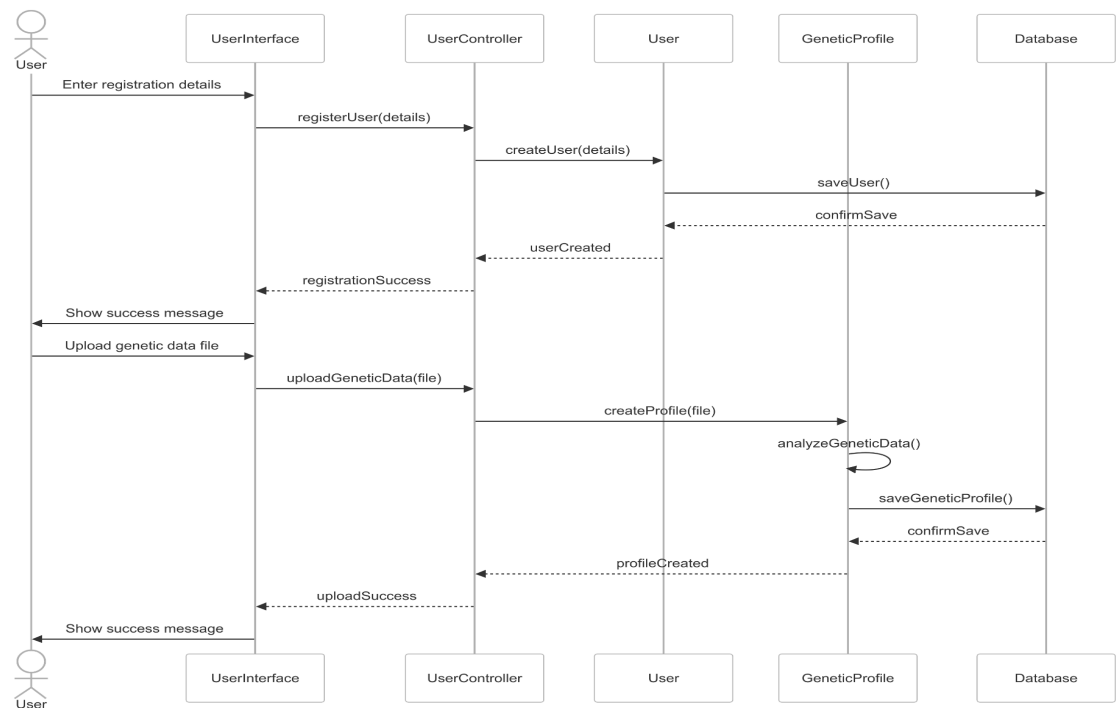


Figure 3.2.3.1 System Sequence Diagram

2. Generating Personalized Meal Plan

This illustrates the process of generating a personalized meal plan based on the user's genetic profile.

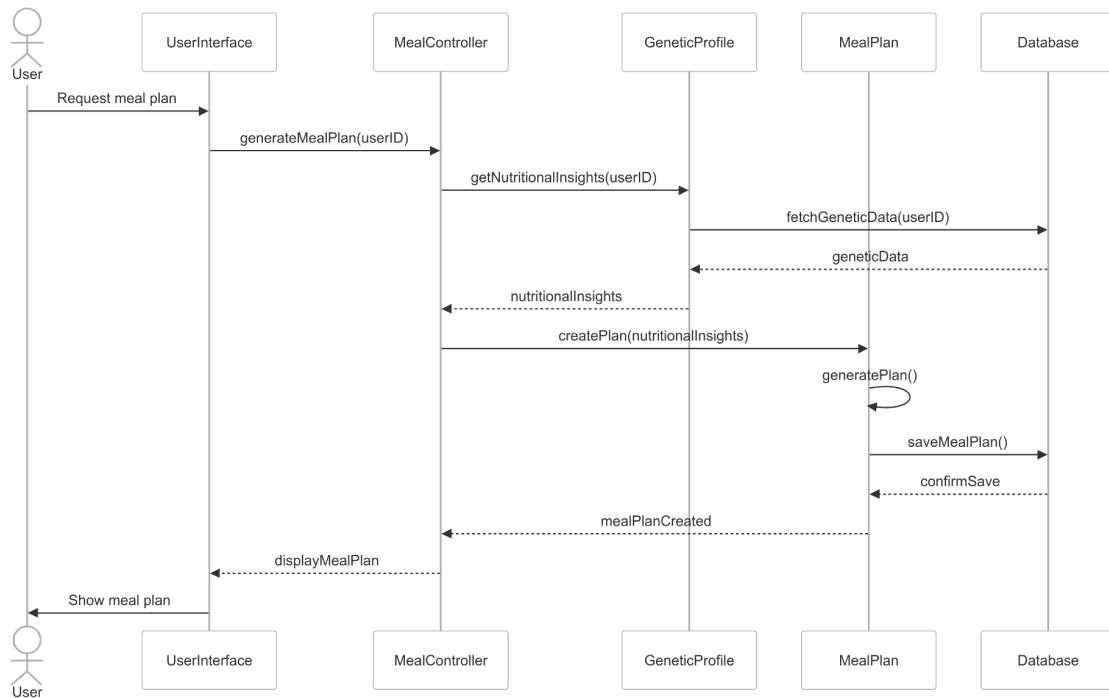


Figure 3.2.3.2 System Sequence Diagram

3. Logging Daily Food Intake

This sequence diagram shows the process of a user logging their daily food intake.

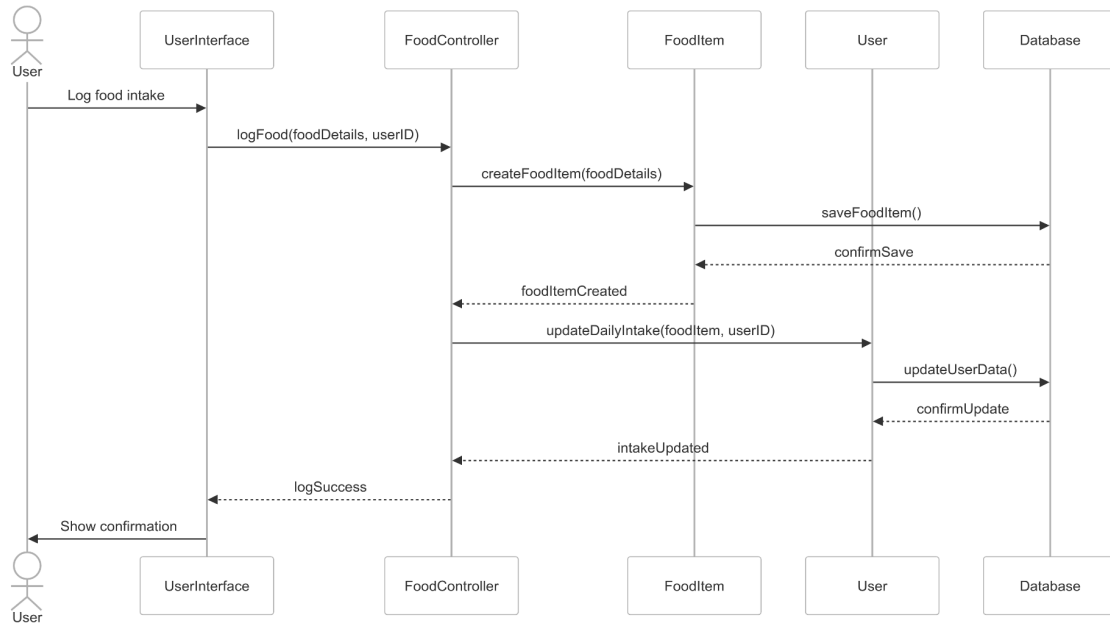


Figure 3.2.3.3 System Sequence Diagram

4. Viewing Health Dashboard

This diagram illustrates the process of fetching and displaying the user's health dashboard.

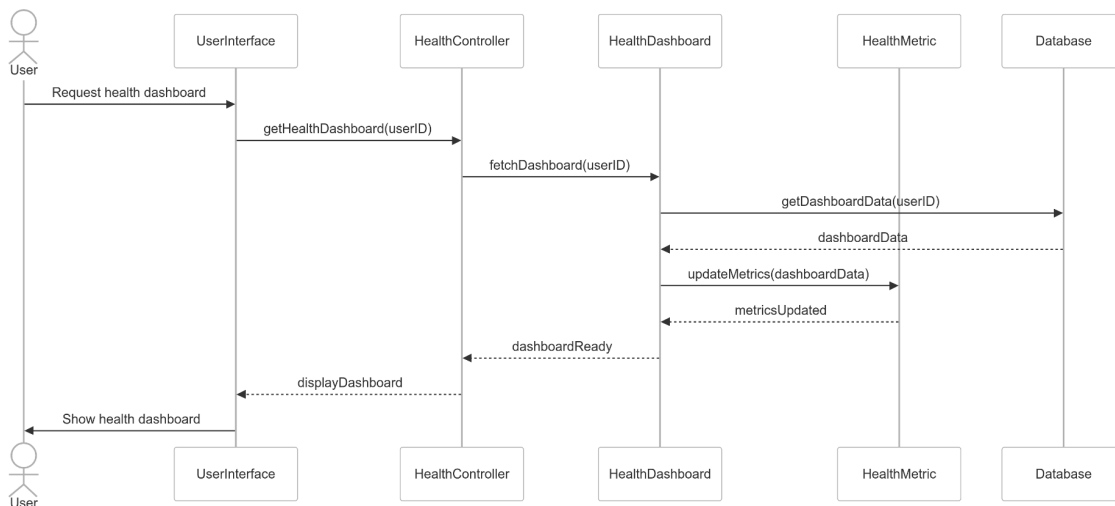


Figure 3.2.3.4 System Sequence Diagram

These System Sequence Diagrams provide a detailed view of how different classes in NutriGen interact to perform key functions. They show the flow of control and data between the user interface, controller classes, model classes, and the database, giving a clear picture of the system's internal workings.

3.2.4 State Transition Diagram

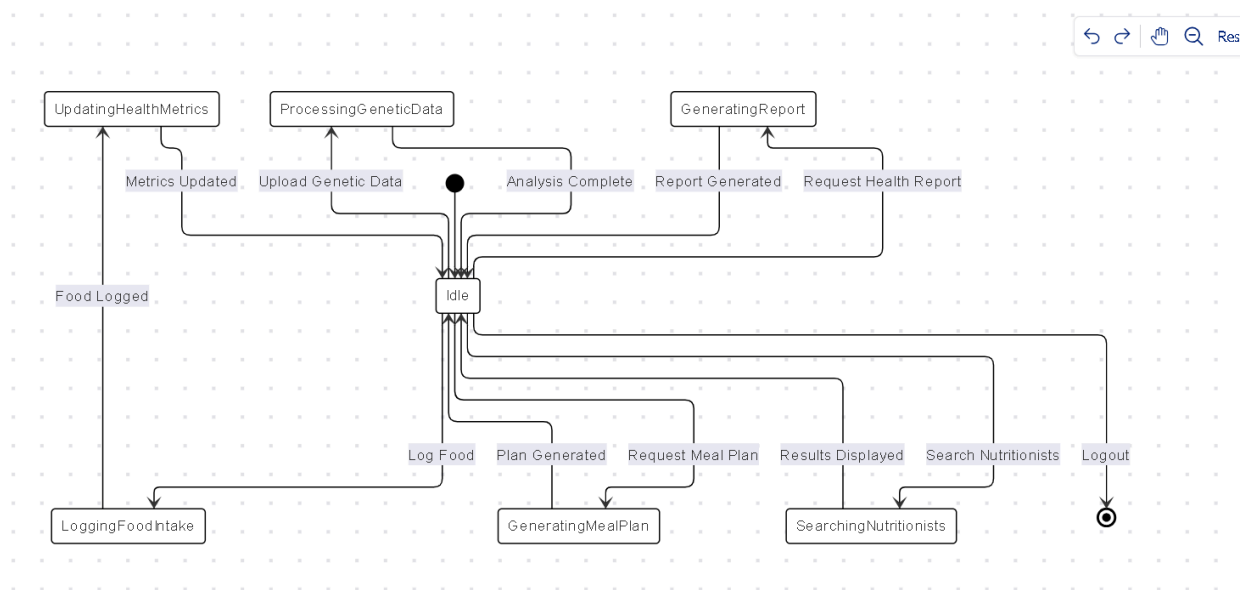


Figure 3.2.4.1 State Transition Diagram

The state transition diagram shows how NutriGen works. It starts in an Idle state, ready for user actions. When users do things like upload genetic data or ask for a meal plan, the system moves to different states to handle these tasks. After finishing each task, it goes back to Idle. This diagram helps us see how the app responds to user actions, processes data, and manages different features like meal planning and health tracking. It shows that NutriGen is always ready to handle user requests and can switch between different tasks smoothly.

3.3 Data Design

In this section, we'll explain how NutriGen organizes and manages its data. We'll cover the main data structures, storage solutions, and how data flows through the system.

3.3.1 Data Structures Overview

NutriGen's data is organized around key entities that represent the core elements of the system. Each entity has specific attributes and relationships with other entities.

3.3.1.1 User Data

- **Attributes:** UserID, Name, Email, PasswordHash
- **Storage:** Secure relational database
- **Use:** User authentication and profile management

3.3.1.2 Genetic Profile Data

- **Attributes:** ProfileID, UserID (Foreign Key), GeneticDataFile, AnalysisResults
- **Storage:** Secure cloud storage for raw data, relational database for analysis results
- **Use:** Basis for personalized nutrition recommendations

3.3.1.3 Meal Plan Data

- **Attributes:** PlanID, UserID (Foreign Key), CreationDate, PlanDetails
- **Storage:** Relational database
- **Use:** Stores personalized meal plans for users

3.3.1.4 Food Item Data

- **Attributes:** FoodID, Name, NutritionInfo, Calories
- **Storage:** Relational database
- **Use:** Building blocks for meal plans and nutritional analysis

3.3.1.5 Health Goal Data

- **Attributes:** GoalID, UserID (Foreign Key), Description, TargetDate, Progress
- **Storage:** Relational database
- **Use:** Tracks user's health objectives and progress

3.3.1.6 Health Metric Data

- **Attributes:** MetricID, UserID (Foreign Key), MetricType, Value, Date
- **Storage:** Relational database
- **Use:** Monitors user's health indicators over time

3.3.1.7 Nutritionist Profile Data

- **Attributes:** NutritionistID, Name, Specialization, ContactInfo

- **Storage:** Relational database
- **Use:** Provides information about available nutritionists

3.3.2 Data Storage Solutions

NutriGen employs a mix of storage solutions to efficiently manage different types of data, this is just an overview and may differ from the implementation:

1. Relational Database (e.g., PostgreSQL):

- Stores structured data like user profiles, meal plans, and health metrics
- Enables efficient querying and data relationships

2. Cloud Storage (e.g., AWS S3):

- Stores large files like genetic data reports
- Provides secure and scalable storage for sensitive information

3. In-Memory Cache (e.g., Redis):

- Caches frequently accessed data like user preferences and recent meal plans
- Improves system performance and response times

3.3.3 Data Flow and Processing

1. User Registration and Genetic Data Upload:

- User info goes into the relational database
- Genetic data file is stored in cloud storage
- Analysis results are saved in the relational database

2. Meal Plan Generation:

- System fetches genetic analysis and user preferences
- Generates meal plan using food item database
- Stores new meal plan in the database

3. Health Tracking:

- User inputs daily food intake and health metrics
- System updates health metric data and goal progress
- Recalculates nutritional recommendations if needed

4. Reporting:

- System compiles data from various entities
- Generates user-friendly reports and visualizations

3.3.4 Data Security and Privacy

- All sensitive data, especially genetic information, is encrypted at rest and in transit
- Access to user data is strictly controlled and logged
- Regular security audits and updates are performed to ensure data protection

This data design ensures that NutriGen can efficiently store, process, and retrieve the information needed to provide personalized nutrition recommendations while maintaining user privacy and data security.

3.3.5 Entity Relationship Diagram

An Entity-Relationship Diagram (ERD) is a visual representation of how data is organized and connected in a system. It shows the main types of information (entities) and how they relate to each other. ERDs help designers and developers understand the structure of a database and how different parts of the system interact with each other.

Key entities in our ERD:

- 1. User:** Represents app users
- 2. GeneticProfile:** Stores user's genetic data
- 3. MealPlan:** Contains personalized meal plans
- 4. FoodItem:** Represents individual food items
- 5. HealthGoal:** Tracks user's health objectives

- 6. HealthMetric:** Records user's health measurements
- 7. Nutritionist:** Stores information about nutritionists
- 8. Allergy:** Tracks user's food allergies
- 9. FoodLog:** Records daily food intake
- 10. Badge:** Represents achievements in the app
- 11. NutritionistReview:** Stores user reviews of nutritionists

Relationships between entities:

- **User** has one **GeneticProfile**
- **User** has many **MealPlans**
- **User** sets many **HealthGoals**
- **User** tracks many **HealthMetrics**
- **User** has many **Allergies**
- **User** records many **FoodLogs**
- **User** earns many **Badges**
- **User** writes many **NutritionistReviews**
- **MealPlan** contains many **FoodItems**
- **Nutritionist** advises many **Users**

Data flow: User uploads genetic data → System generates meal plans → User logs food and tracks health → System updates goals and metrics → User earns badges and reviews nutritionists.

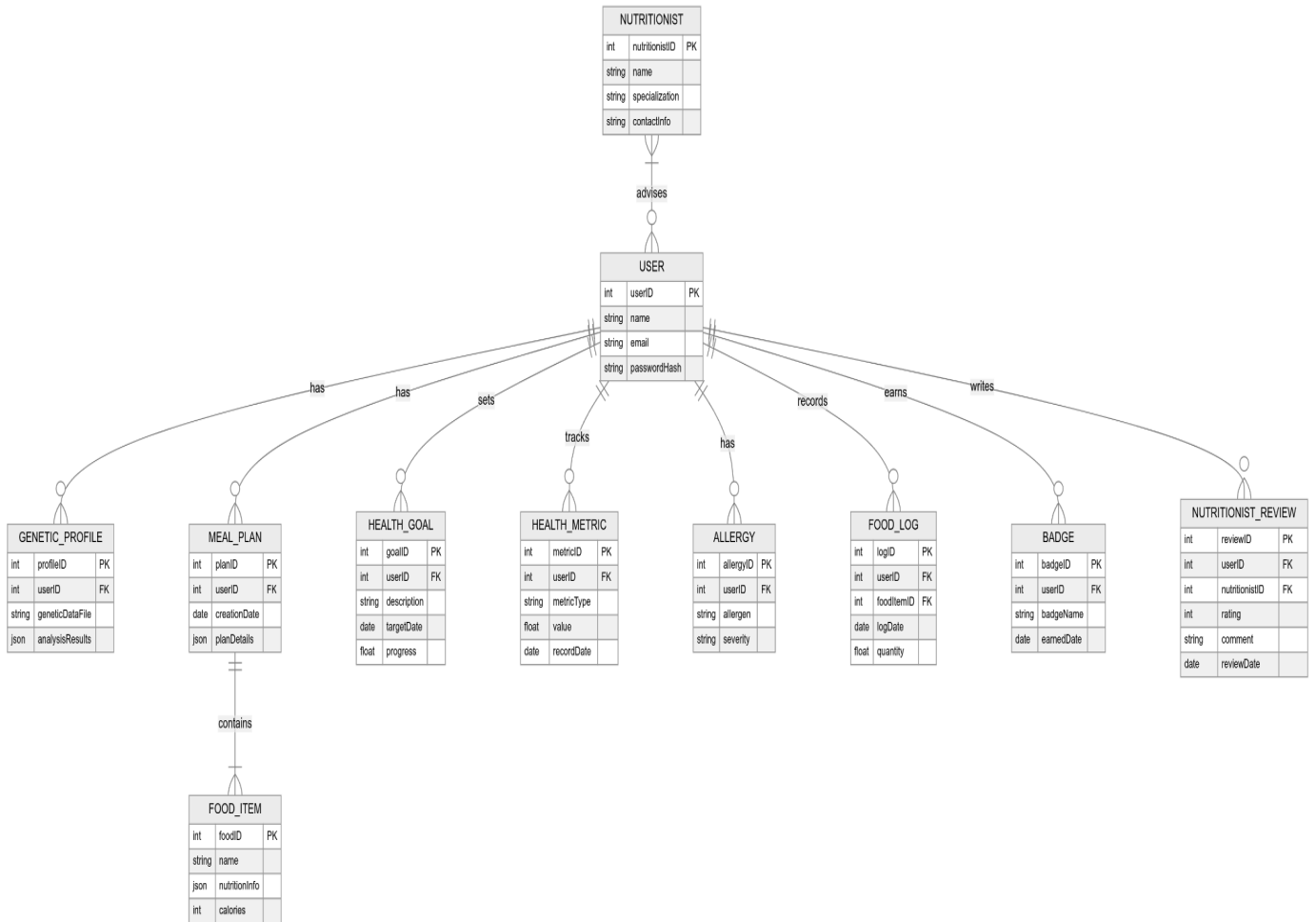


Figure 3.3.5.1 ERD

The ERD for NutriGen shows how different parts of the app work together. It centers around the User, who has a genetic profile, meal plans, and health goals. Users can log their food, track health metrics, and manage allergies. The system creates personalized meal plans using food items based on the user's genetic data and goals. Users can also interact with nutritionists, earning badges for progress and leaving reviews. This structure allows NutriGen to offer personalized nutrition advice, track user progress, and

provide a comprehensive health management experience, all while keeping user data organized and connected.

Chapter 4: Conclusions and Future Work

4.1 Project Summary

We've planned NutriGen, an app to help people eat better using their genetic information. Our design includes:

1. A way to look at genetic data and suggest good foods
2. A system to make personalized meal plans
3. Tools for tracking what people eat and their health
4. A part where people can find food experts
5. Fun features like badges to keep people interested

We think NutriGen could help many people eat in a way that's right for their bodies.

4.2 Next Steps

To make NutriGen work, we need to:

1. Find the right tools to analyze genetic data
2. Create a good chatbot to help users
3. Build a safe way to store people's information
4. Make the app easy for people to use
5. Test the app to make sure it works well

4.3 Future Ideas

In the future, we may:

1. Make the app work with fitness trackers
2. Add more languages
3. Work with doctors to improve our advice

4.4 Closing Thoughts

NutriGen is a new idea in using genetic information for better eating. While there's still much to do, we're excited about how it could help people be healthier.